KENNESAW STATE
UNIVERSITY

CS 7357
NEURAL NETS AND DEEP LEARNING

ASSIGNMENT 4

QUANTUM MACHINE LEARNING LAB M0

INSTRUCTOR

Dr. Zongxing Xie

Sara Aliaga

000835610

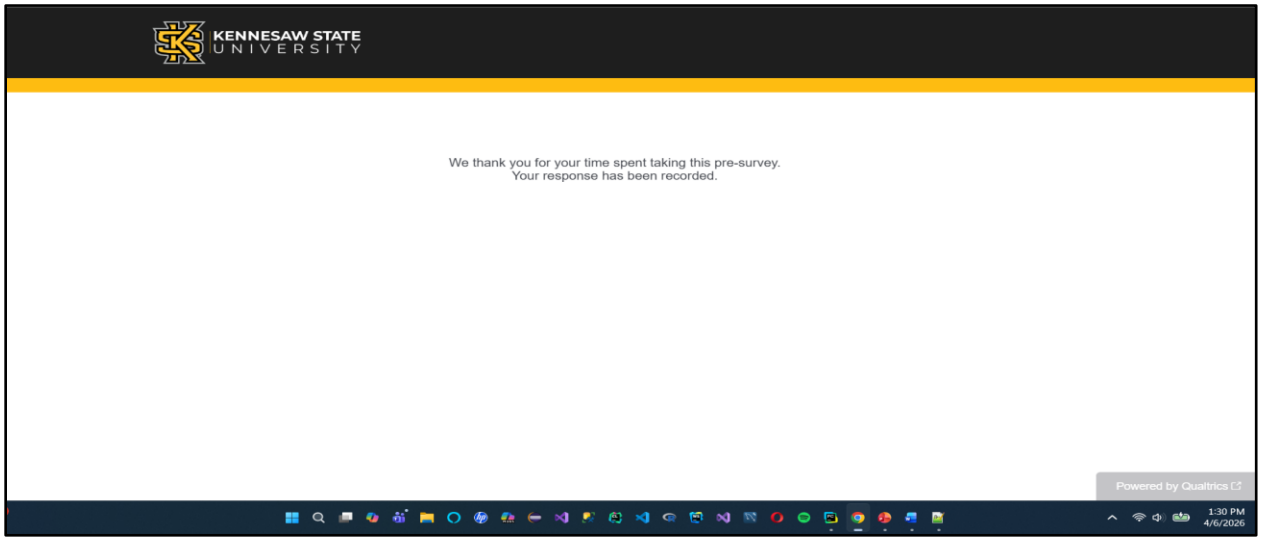
Introduction

This lab focuses on the fundamentals of Quantum Machine Learning (QML) by combining concepts from quantum computing and classical machine learning. The objective of Lab M0 is to explore how quantum circuits can be used to approximate mathematical functions using a single qubit and parameterized quantum gates. In this lab, we implemented quantum models to represent different functions, including square, cosine, and polynomial functions.

The lab is divided into multiple tasks to demonstrate different approaches to training quantum models. In Task 1, we constructed quantum circuits and evaluated their performance without using any optimization method. In Task 2, we improved the model accuracy by applying a Gradient Descent Optimizer to adjust circuit parameters. In Task 3, we further explored optimization by manually computing gradients and updating parameters. Finally, in Task 4, we selected a custom mathematical function and implemented it using a quantum circuit without an optimizer.

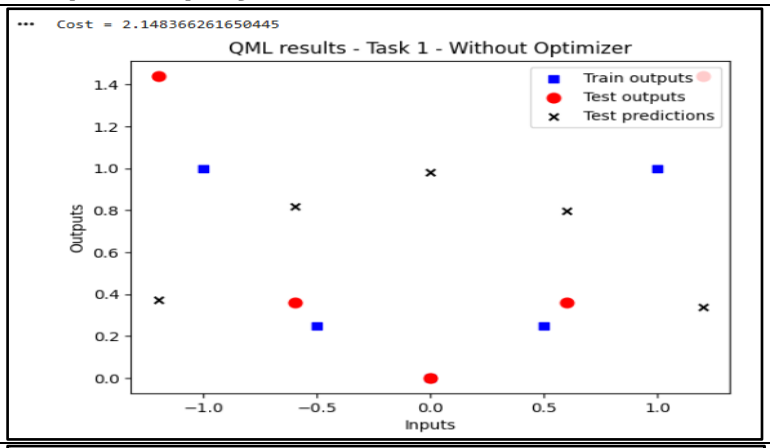
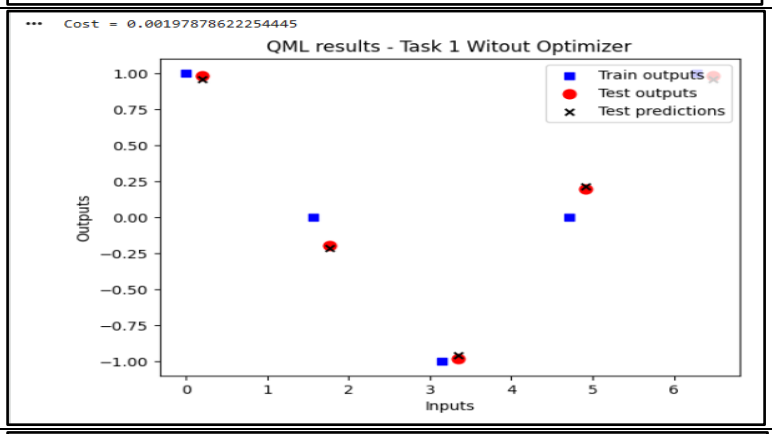
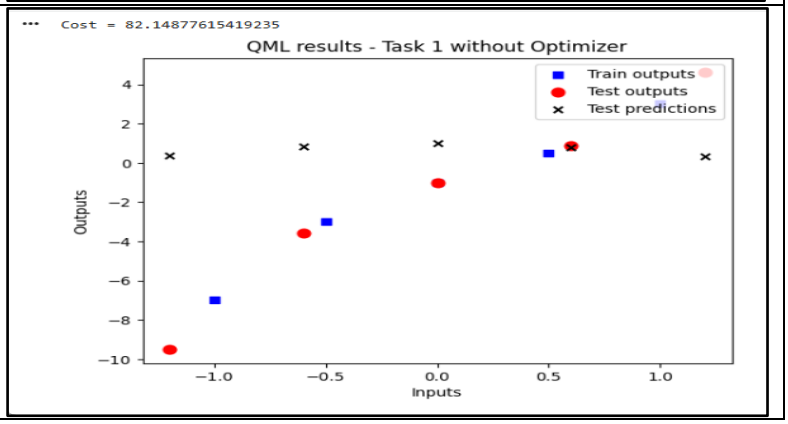
Through this lab, we gained hands-on experience with PennyLane, learned how to design and evaluate quantum circuits, and understood how optimization techniques influence model performance in quantum machine learning.

Pre-survey - Tutorial M0 (Screenshot)



Main Assignment

Task 1 - Without Optimizer - Result after making changes

Function Name	Output display – Task 1
Square function Without Optimizer	
Cosine function Without Optimizer	
Polynomial function Without Optimizer	

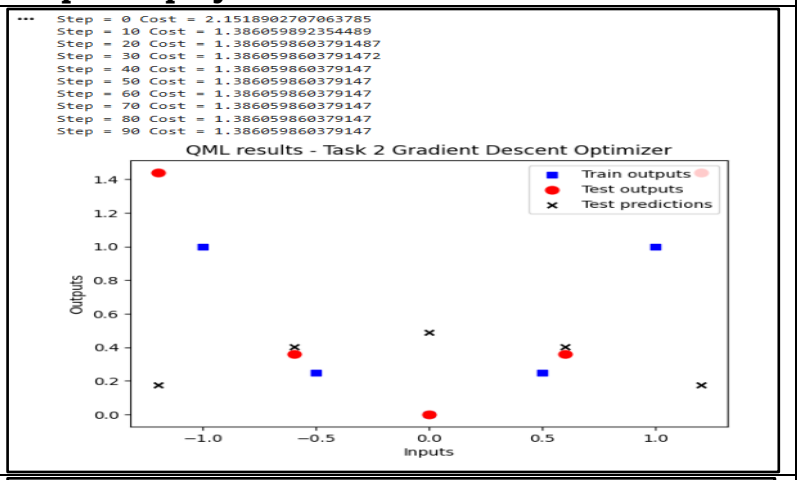
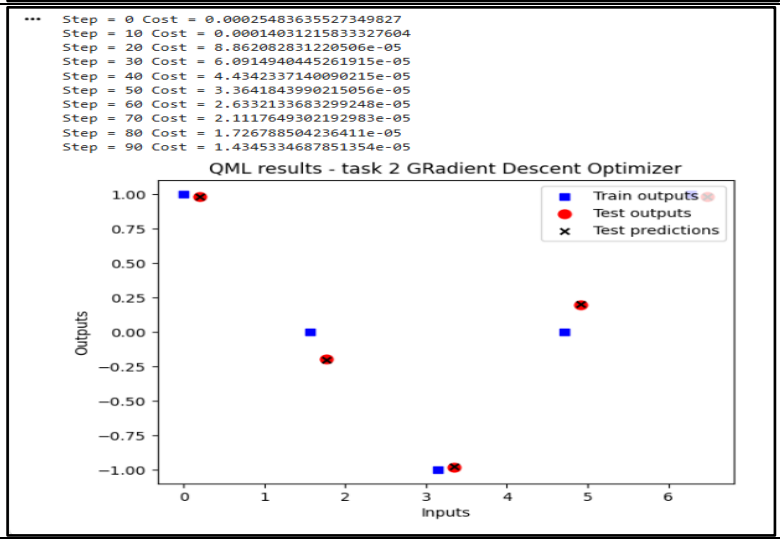
Description

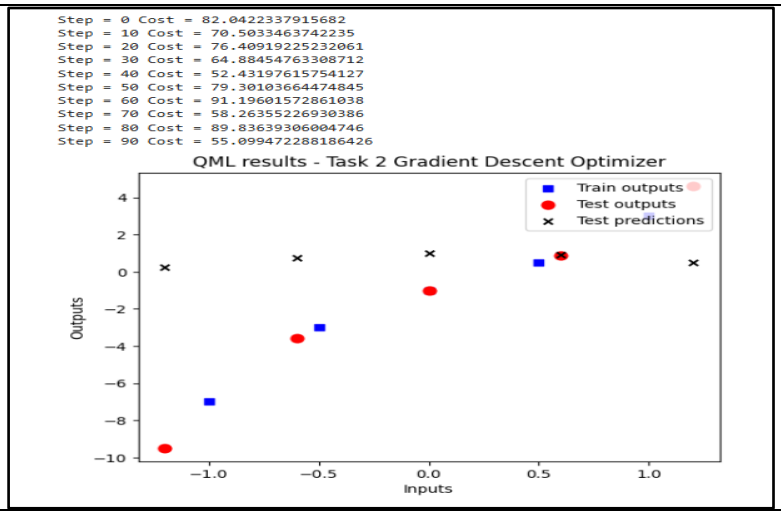
Task 1 focuses on implementing quantum circuits **without using any optimization technique.** In this task, fixed parameter values were assigned to the quantum rotation gates, and no learning process was applied.

The goal was to evaluate how well a quantum circuit can approximate different mathematical functions (square, cosine, and polynomial) using only initial parameter settings. The input data

was encoded into the quantum circuit using an RX gate, followed by a parameterized rotation gate. The output was obtained by measuring the expectation value of the Pauli-Z operator. The results show that without optimization, the quantum model produces predictions that are not well aligned with the true function values, demonstrating the importance of parameter tuning in quantum machine learning.

Task 2 with Gradient Descent Optimizer – After making changes

Function Name	Output display - Task 2
Square Function with Gradient Descent Optimizer	
Cosine Function with Gradient Descent Optimize	

Polynomial Function with Gradient Descent Optimizer	

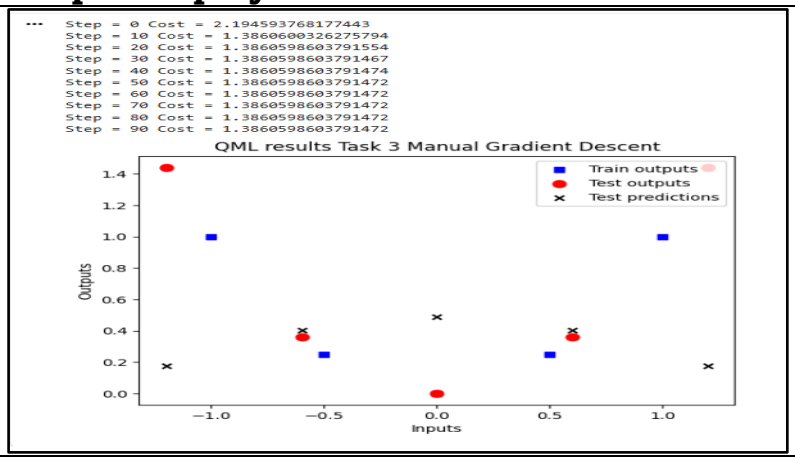
Description

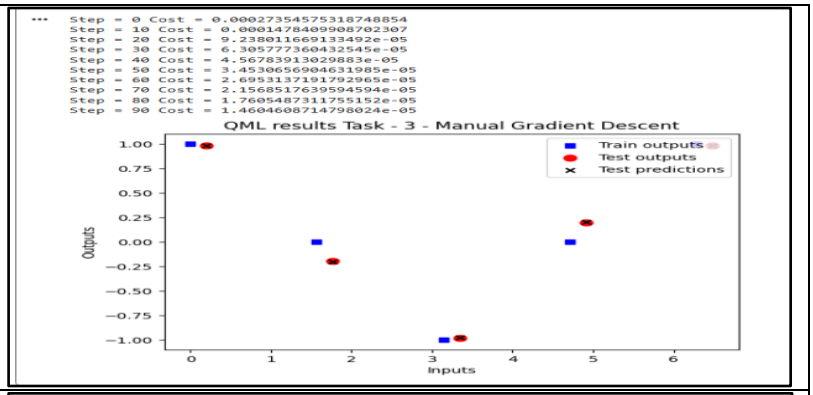
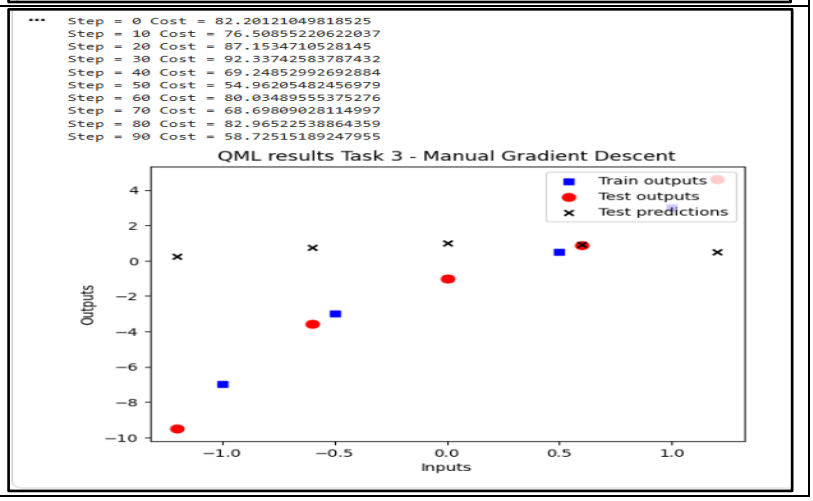
In Task 2, a **Gradient Descent Optimizer** was introduced to improve the performance of the quantum model. Unlike Task 1, the parameters of the quantum circuit were updated iteratively to minimize the loss function.

The optimizer adjusted the rotation gate parameters using automatic differentiation provided by PennyLane. Over multiple iterations, the model learned to better approximate the target functions (square, cosine, and polynomial).

The results show a clear improvement compared to Task 1, as the predicted output more closely matches the actual values. This demonstrates how optimization significantly enhances the learning capability of quantum circuits.

Task 3 – Manual Gradient Descent Optimizer – After Changes

Function Name	Output display - Task 3
Square function with Manual Gradient Descent Optimizer	

Cosine function with Manual Gradient Descent Optimizer	
Polynomial function with Manual Gradient Descent Optimizer	

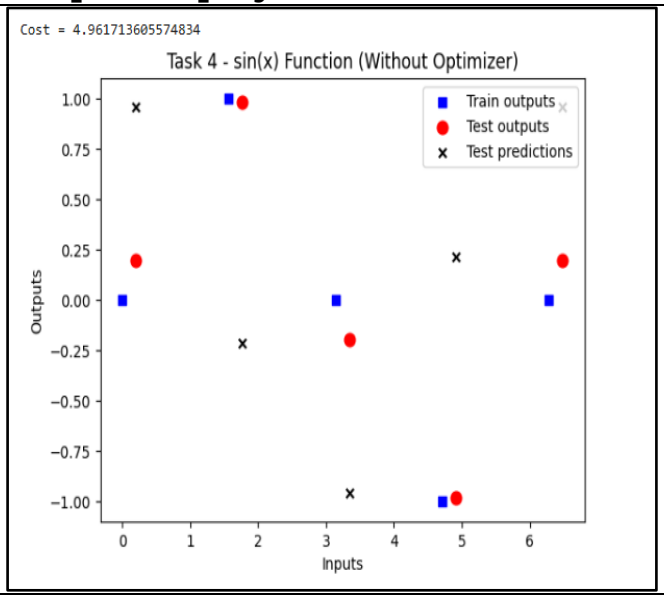
Description

Task 3 explores a deeper understanding of optimization by implementing **manual gradient descent** instead of using a built-in optimizer.

In this task, gradients of the cost function were computed manually using qml.grad(), and parameters were updated step-by-step using a defined learning rate. This approach provided insight into how optimization works internally in quantum machine learning models.

The results were similar to Task 2, showing improved accuracy compared to Task 1. However, this task emphasized the learning process itself, helping to understand how gradients influence parameter updates and model performance.

Task 4 - Create my own Function without Optimizer

Function Name	Output Display
SIN(x) Function without Optimizer	

Main Assignment - Task 4 - SIN(x) Function Without optimizer

Source code for Task 4

```
#TASK 4 - MY OWN FUNCTION (sin(x)) WITHOUT OPTIMIZER
# Import the necessary libraries
import pennylane as qml                  # for quantum computing operations
from pennylane import numpy as np       # PennyLane-compatible NumPy
import matplotlib.pyplot as plt         # for plotting graphs
# Define my own mathematical function
def my_function(x):
    return np.sin(x)
# Create the training and test data
X = np.linspace(0, 2*np.pi, 5)          # 5 input datapoints from 0 to 2pi
X.requires_grad = False
Y = my_function(X)                      # outputs for the training datapoints

X_test = np.linspace(0.2, 2*np.pi + 0.2, 5)   # shifted test datapoints
Y_test = my_function(X_test)                  # outputs for the test datapoints

# Setting up Quantum Device
# Create the quantum device with 1 qubit
dev = qml.device('default.qubit', wires=1)
# Create the quantum circuit
@qml.qnode(dev)
def quantum_circuit(datapoint, params):
    # Encode the input data using an RX rotation
    qml.RX(datapoint, wires=0)
    # Apply a rotation gate based on three parameters
    qml.Rot(params[0], params[1], params[2], wires=0)

    # Return the expected value of a measurement along the Z axis
    return qml.expval(qml.PauliZ(wires=0))
# Define the loss function
def loss_func(predictions):
    total_losses = 0
    for i in range(len(Y)):
        output = Y[i]
        prediction = predictions[i]
        loss = (prediction - output)**2
        total_losses += loss
    return total_losses

# Define the cost function
def cost_fn(params):
    predictions = [quantum_circuit(x, params) for x in X]
    cost = loss_func(predictions)
    return cost
# Fixed parameters (no optimizer used)
params = np.array([0.1, 0.2, 0.3], requires_grad=True)
# Print the cost
print("Cost =", cost_fn(params))
# Test and graph the results
test_predictions = [quantum_circuit(x_test, params) for x_test in X_test]

fig = plt.figure()
ax1 = fig.add_subplot(111)
ax1.scatter(X, Y, s=30, c='b', marker="s", label='Train outputs')
ax1.scatter(X_test, Y_test, s=60, c='r', marker="o", label='Test outputs')
ax1.scatter(X_test, test_predictions, s=30, c='k', marker="x", label='Test predictions')

plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.title("Task 4 - sin(x) Function (Without Optimizer)")
plt.legend(loc='upper right')
plt.show()
```

Description:

In Task 4, a custom function (**sin(x)**) was implemented using a quantum circuit without applying any optimizer.
The purpose of this task was to test the flexibility of quantum circuits in modeling new functions beyond those provided in the tutorial. Similar to Task 1, fixed parameters were used, and no training process was applied.
The results showed that while the quantum circuit could generate output, the predictions did not closely match the true sine function due to the absence of optimization. This highlights that optimization is essential for achieving accurate function approximation in quantum models.

Post – Lab M0 - sin(x) with gradient descent optimizer, loss(), cost(), quantum device, train and test

In the post-Lab, the sine function sin $(x)$was implemented as a custom mathematical function using a quantum circuit and trained with the Gradient Descent Optimizer. Unlike the Task 4 version without optimization, this model updates the circuit parameters iteratively to minimize the cost function. The optimized results show improved predictions compared to the non-optimized version, demonstrating the importance of gradient descent in training quantum machine learning models

Output

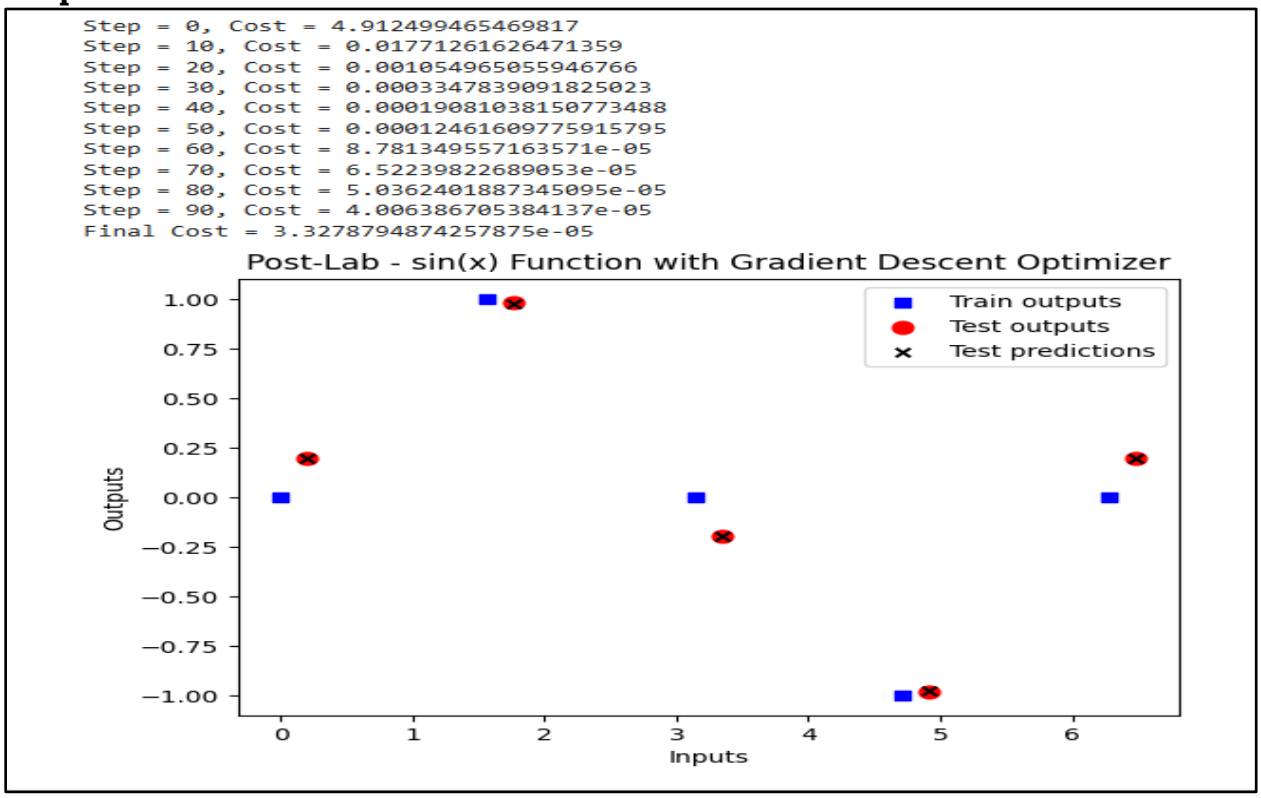


Figure 1. Post-Lab implementation of the custom sine function using a quantum circuit with Gradient Descent Optimizer. The graph compares the training outputs, test outputs, and optimized test predictions.

Source Code Post – Lab (M0)

```
# POST-LAB - MY OWN FUNCTION (sin(x)) WITH GRADIENT DESCENT OPTIMIZER

# Import the necessary libraries
import pennylane as qml                  # for quantum computing operations
from pennylane import numpy as np       # PennyLane-compatible NumPy
import matplotlib.pyplot as plt         # for plotting graphs

# Define my own mathematical function
def my_function(x):
    return np.sin(x)

# Create the training and test data
X = np.linspace(0, 2*np.pi, 5)          # 5 input datapoints from 0 to 2pi
X.requires_grad = False
Y = my_function(X)                      # outputs for the training datapoints

X_test = np.linspace(0.2, 2*np.pi + 0.2, 5)   # shifted test datapoints
Y_test = my_function(X_test)                  # outputs for the test datapoints

# Setting up Quantum Device
# Create the quantum device with 1 qubit
dev = qml.device('default.qubit', wires=1)

# Create the quantum circuit
@qml.qnode(dev)
def quantum_circuit(datapoint, params):
    # Encode the input data using an RX rotation
    qml.RX(datapoint, wires=0)

    # Apply a rotation gate based on three parameters
    qml.Rot(params[0], params[1], params[2], wires=0)

    # Return the expected value of a measurement along the Z axis
    return qml.expval(qml.PauliZ(wires=0))

# Define the loss function
def loss_func(predictions):
    total_losses = 0
    for i in range(len(Y)):
        output = Y[i]
        prediction = predictions[i]
        loss = (prediction - output)**2
        total_losses += loss
    return total_losses

# Define the cost function
def cost_fn(params):
    predictions = [quantum_circuit(x, params) for x in X]
    cost = loss_func(predictions)
    return cost

# Define the optimizer
opt = qml.GradientDescentOptimizer(stepsize=0.3)

# Initial guess for the parameters
params = np.array([0.1, 0.1, 0.1], requires_grad=True)

# Optimization loop
for i in range(100):
```

```
    params, prev_cost = opt.step_and_cost(cost_fn, params)
    if i % 10 == 0:
        print(f"Step = {i}, Cost = {cost_fn(params)}")

# Print final cost
print("Final Cost =", cost_fn(params))

# Test and graph the results
test_predictions = [quantum_circuit(x_test, params) for x_test in X_test]

fig = plt.figure()
ax1 = fig.add_subplot(111)

ax1.scatter(X, Y, s=30, c='b', marker="s", label='Train outputs')
ax1.scatter(X_test, Y_test, s=60, c='r', marker="o", label='Test outputs')
ax1.scatter(X_test, test_predictions, s=30, c='k', marker="x", label='Test predictions')

plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.title("Post-Lab - sin(x) Function with Gradient Descent Optimizer")
plt.legend(loc='upper right')
plt.show()
```

Post – lab Analysis (M0)

In the post-Lab, the sine function sin $(x)$was implemented using a quantum neural network and trained with a Gradient Descent Optimizer. Compared to the non-optimized version from Task 4, the optimized model showed a significant improvement in performance. The predictions generated by the quantum circuit more closely followed the true sine curve, indicating that the model successfully learned the underlying pattern of the function.
During training, the cost function decreased over iterations, demonstrating that the optimizer effectively adjusted the circuit parameters to minimize error. The alignment between test outputs and predicted values confirms that optimization plays a critical role in improving the accuracy of quantum models.
This experiment highlights that while quantum circuits can represent mathematical functions, they require proper training to achieve meaningful results. The use of gradient descent enables the quantum model to generalize better and produce more reliable predictions.

Conclusion

In this lab, we explored the fundamentals of Quantum Machine Learning by implementing quantum circuits to approximate different mathematical functions. Through Tasks 1 to 4, we learned how to construct quantum models, encode data using quantum gates, and evaluate outputs using expectation values.
The progression from models without optimization to models using Gradient Descent demonstrated the importance of parameter tuning in improving model performance. Additionally, implementing manual gradient descent provided deeper insight into how optimization algorithms work internally.
The Post-Lab further reinforced these concepts by applying a trained quantum model to a new function, showing how optimization enhances prediction accuracy and model reliability. Overall, this lab provided valuable hands-on experience with PennyLane and strengthened our understanding of how quantum computing principles can be applied to machine learning tasks.

Post-survey - Tutorial M0 (Screenshot)